Feature Extraction and Latent Space Analysis of MNIST Using a Custom Autoencoder

Kevin Nguyen
Student ID: 230085567

May 19, 2026

Abstract

This report details the implementation and analysis of a custom auto encoder neural network built using PyTorch. The network was trained on the standard MNIST dataset of handwritten digits to perform dimensionality reduction and feature extraction. By compressing the images down to a 3-dimensional space, the model successfully learned the fundamental structural features of the digits. Performance was evaluated through visual reconstruction fidelity and morphing and looking at the loss function, demonstrating the network's ability to generate continuous, non-existent data points.

1 Task Description

The primary objective of this project is to build an auto encoder, train it using the MNIST dataset, and analyse the extracted features. An auto encoder is an unsupervised artificial neural network trained to copy its input to its output. It achieves this by utilizing an "hourglass" architecture, consisting of an encoder that compresses the data into a lower-dimensional latent space (the bottleneck), and a decoder that reconstructs the original data from this compressed representation.

The MNIST dataset, that i decided to use consisted of 60,000 training images and 10,000 testing 28x28 gray scale images of handwritten digits for this task. The main of using an auto encoder on this dataset is not simply to memorise the images, but to force the network to identify and extract the most critical geometric features (such as curves, loops, and angles) required to represent a digit in the dataset. By deliberately restricting the bottleneck to just 3 neurons, the network is forced to learn a highly compressed, continuous feature map. This is then tested to see if we can reconstruct a digit from the original. I decided to use this data set from seeing it in week 9 on our lab.

2 Architecture and Training Strategy

2.1 Network Architecture

The auto encoder was implemented using PyTorch's `nn.Module` and utilises standard Fully Connected (Linear) layers, which are highly efficient for centred, gray scale datasets like MNIST.

- **Encoder:** The encoder takes the flattened 784-pixel input array and passes it through two hidden layers of 128 and 64 neurons, utilising ReLU (Rectified Linear Unit) activation functions, this introducing non-linearity. The final layer compresses the data into a 3-neuron bottleneck, representing the extracted latent features. This is a heavy compression,

but it ensures that the model is not memorising the handwriting and is learning how to construct it.

- **Decoder:** The decoder mirrors this structure, taking the 3-dimensional latent vector and expanding it through hidden layers of 64 and 128 neurons (with ReLU activations). The final output layer maps back to 784 neurons. A Sigmoid activation function is applied at the very end to ensure the output pixel values are bounded between 0.0 and 1.0, matching the normalized input data.
  - ReLU is the usual standard rather than something like sigmoid as the activation function for hidden layers in order to prevent the vanishing gradient problem (derivative of ReLU always 1) from occurring during back propagation.

2.2 Backpropagation and Weight Optimization

The auto encoder learns through back propagation, which computes the gradient of the loss function with respect to the network's weights using the chain rule of calculus. In this implementation, PyTorch's `autograd` engine handles this automatically. Calculating the MSE loss and calling `loss.backward()` triggers the backward pass, which traverses the computational graph in reverse to find the necessary partial derivatives.

The network's weights are then updated using the Adam optimiser, chosen over standard SGD for its adaptive learning rates and faster, more stable convergence. Crucially, `optimizer.zero_grad()` must be called before each batch. This clears the gradients from the previous step, preventing them from accumulating and corrupting the network's learning process. I decided to use Adam optimiser from seeing it in our lab in week 10 where we were using a MNIST data set too.

2.3 Training Strategy

The model was trained using the following parameters and strategies:

- **Loss Function:** Mean Squared Error (`nn.MSELoss`) was selected because the task requires calculating the direct pixel-by-pixel difference between the original input image and the reconstructed output image. When finding the loss from each epoch we are expecting the MSE loss to get smaller meaning that the model is 'getting better' or a reconstruction of the image being closer to the original image.
- **Optimiser:** The Adam optimiser (`optim.Adam`) was utilised with a learning rate of 0.001 (we use a small learning rate to not miss the optimal solution) to ensure efficient and adaptive gradient descent. We use Adam optimiser as it uses the moment of the previous gradient to speed up the training.
- **Batching and Epochs:** The training data was loaded using PyTorch's `DataLoader` with a batch size of 64, shuffling the data every epoch to prevent the model from memorising the sequence. The model was trained for 10 epochs. We use only 10 epoch so that over-fitting doesn't occur, as if we did, say 100 epochs the model reaches a convergence point for the test data, this could mean that its not actually reconstructing the images.
- **Validation Loss:** We are now applying loss function to the unseen test dataset rather than training, we want the results to mirror the training loss, indicating that that the model is well generalised and to see if over-fitting was prevented.

3 Analysis of the Results

3.1 Reconstruction Fidelity

To evaluate the network's baseline performance, unseen images from the test dataset were passed through the fully trained model.

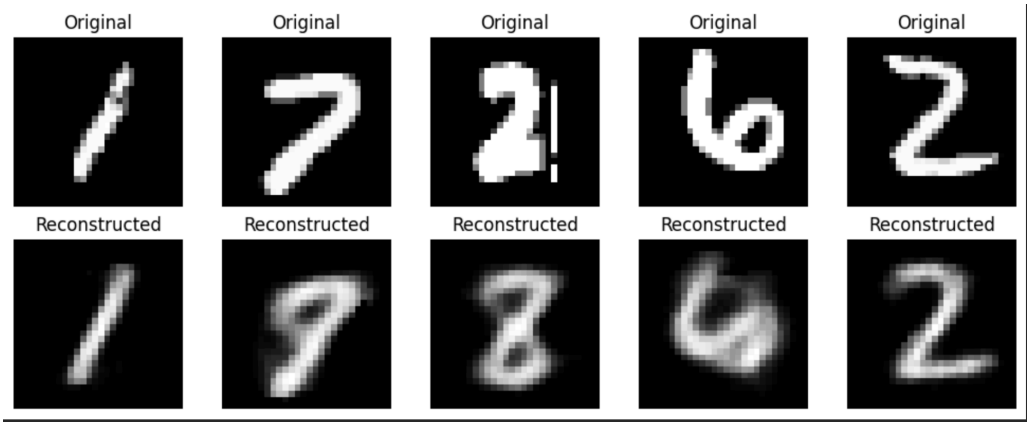

Figure 1: Top row: Original MNIST digits. Bottom row: Auto encoder reconstructions.

As seen in Figure 1, the network successfully reconstructs the general shape of the digits. Because the network was forced to compress 784 pixels of information into only 3 numbers, there is an expected loss of high-frequency detail, resulting in slight blurring. However, the core geometric features of each digit are preserved and clearly recognizable, proving the encoder successfully captured the vital data. We can see this from the 1 6 and 2 having the same structure, even though the original 7 was reconstructed to a 9 (likely from high compression) we can see the overlapping geometric shape, this also shows the model is learning rather than memorising.

3.2 Loss Function and Training Convergence

To measure the auto encoder's performance during training, the Mean Squared Error (MSE) loss function was utilized. Since the goal of the network is to output an image that matches the input as closely as possible, MSE is ideal because it calculates the average squared difference between the original pixel values and the reconstructed pixel values. Mathematically, it is defined as:

$$\mathcal{L}_{MSE} = \frac{1}{N}\sum_{i=1}^{N}(x_i - \hat{x}_i)^2$$

where N is the total number of pixels in the image (784), x_i represents the true value of pixels from the original MNIST image, and $\hat{x}_i$ represents the predicted value of pixels generated by the decoder.

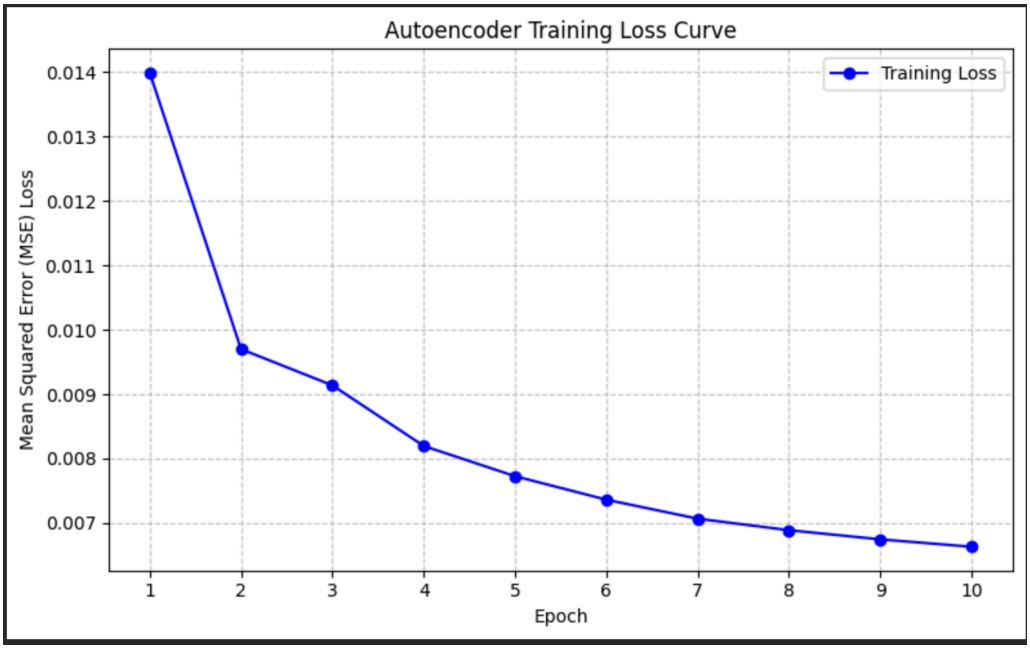

Figure 2: Training loss curve demonstrating the decrease in Mean Squared Error over 10 epochs.

As illustrated in Figure 2, the model demonstrated excellent convergence over the 10 training epochs. The initial loss in Epoch 1 started approximately 0.014. During the first three epochs, the network experienced rapid learning, characterized by a steep decline in the error rate. By Epoch 10, the loss smoothly converged to roughly 0.0067. This smooth, asymptotic curve confirms that the Adam optimizer effectively navigated the gradient space without oscillating, successfully minimizing the reconstruction error. We could have done more Epochs' ,but the difference would be minimal in this case and would increase the amount of compute. This amount of compute would be emphasise with a larger model.

3.3 Extracted Features: Latent Space Interpolation

To truly analyse the extracted features, we bypass the reconstruction of existing images and instead evaluate the continuous nature of the latent space via interpolation (morphing).

We extracted the 3-dimensional latent coordinates of a '9' and a '1'. We then mathematically interpolated a straight line between these two points in the latent space, generating 15 evenly spaced coordinate vectors. These non-existent, transitional vectors were then fed directly into the decoder.

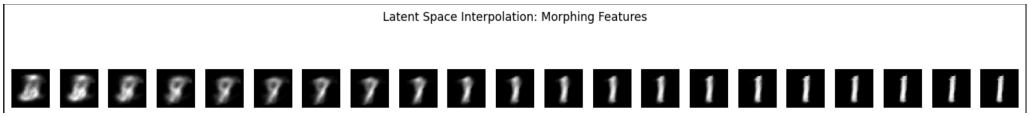

Figure 3: Latent space interpolation showing the structural morph between a '9' and a '1'.

Figure 3 demonstrates that the auto encoder did not simply memorise discrete images, but rather learned a continuous map of handwriting features. As the vector traverses the latent space, the structural features smoothly bend, we can see the removal of the curves of the '9' only leaving the features of the '1' it has. This confirms the network successfully learned to extract and manipulate the fundamental geometric building blocks of the dataset rather then memorising it learnt.

3.4 Loss Function and Training Convergence

When comparing the training loss and the validation loss, the training loss was 0.0330 whereas, the validation loss gave 0.0333, we can see that there are negligible difference, likely from the addition of a new variation of the test data and from a 3-neuron bottleneck. Anyhow, from the value of the validation loss we can see that there are a few blurry pixels but kept most the core structural information. The values being similar also shows robustness as the model reliability created new images with new data.

4 Conclusion and Own Contribution

This project successfully implemented a dimensionality-reducing auto encoder. While standard PyTorch libraries (`torchvision`, `nn.Module`) were utilized for data loading and backpropagation as permitted, the specific architectural design, the choice of a highly restricted 3-neuron bottleneck to force aggressive feature extraction, the hyper-parameter tuning, and the logic required to mathematically map and visualize the latent space interpolation were my own contributions.